GCSE Computer Science · Assessment 02

Control Systems Feedback

Jacob

10CS · 2026-10-16

Developing · confirmed



SCORE	PERCENTAGE	NOT YET	CHECKPOINT
24 / 54	44.4%	30	Developing

Assessment breakdown

A · Knowledge & tracing	6 / 8	2 not yet
B · Debug lab	0 / 6	6 not yet
C1 · Gaming rank reward	5 / 6	1 not yet
C2 · Greenhouse ventilation	6 / 7	1 not yet
C3 · Parcel sorter	7 / 9	2 not yet
C4 · Digital marketplace	0 / 18	18 not yet

A message from your teacher

Good effort on this assessment Jacob.

You've demonstrated a good understanding of if/elif/else statements, and how comparisons and Boolean operations are used.

You need to take more care with the logical operation (> or <) you select, and with how you order/execute multiple related conditional statements.

WWW · What went well

Nested selection
You met these requirements: Use nested selection for routing within valid-weight processing. Give the output Consent required.
Evidence: C3.8, A4.1.

Logical operators and compound conditions
You met these requirements: Set HIGH when temperature >=30 and humidity >=80, using and. Set LOW when temperature >=30 but humidity is below 80.
Evidence: C2.1, C2.2.

Selection and tracing
You met these requirements: Add Rising for XP from 1000 to 2499 inclusive. Retain the correct Starter, Pro and Elite ranks and rewards.
Evidence: C1.1, C1.3.

Debugging and modification
You met these requirements: Add Rising for XP from 1000 to 2499 inclusive. Assign 250 reward coins to the Rising route.
Evidence: C1.1, C1.2.

EBI · Even better if

Annotation and readability
Focus on these requirements: Explain in a purposeful comment why the highest XP threshold is checked first. Explain in a purposeful comment why HIGH is checked before LOW. Improve a requested comment so it explains why a decision is needed or why routes are ordered that way. Add useful section comments without narrating every line.
Evidence: C1.6, C2.7.

Inputs, calculations and output
Focus on these requirements: Collect customer name, product type and membership with clear prompts. Collect quantity and convert it to an integer. Check a member buying four bundles: subtotal £32.00, discount £4.80, total £27.20. Then check a non-member. Repair the calculation or missing receipt detail and explain the change.
Evidence: C4.1, C4.2.

Validation and invalid routes
Focus on these requirements: Reject weight above 20 kg as REJECTED WEIGHT. Prevent rejected weights from entering destination routing. Test marketplace quantities 0, 1, 10 and 11 and an invalid product. Correct the structure so invalid orders display a rejection without calculating a receipt.
Evidence: C3.2, C3.3.

DIRT · To complete

Improve B1 · Age and consent. You earned 0 / 3 marks. Start with: Prevent approval for ages below 13. Prevent approval for ages above 17. Require consent == "yes" alongside a valid age. Then run suitable inputs from the question and check the output. Work through the remaining checklist requirements before submitting your improved code.

Improve C3 · Parcel sorter upgrade. You earned 7 / 9 marks. Start with: Reject weight above 20 kg as REJECTED WEIGHT. Prevent rejected weights from entering destination routing. Then run suitable inputs from the question and check the output. Work through the remaining checklist requirements before submitting your improved code.

Complete C4 · Digital marketplace order. The submitted code still matches the starter. Start with: Collect customer name, product type and membership with clear prompts. Collect quantity and convert it to an integer. Reject quantities below 1. Then run suitable inputs from the question and check the output. Work through the remaining checklist requirements before submitting your improved code.

Progress rubric

The coloured cell is the teacher-selected level. A skill can remain not assessed where there is not enough evidence.

Comparisons and Boolean results		Developing
✓ **Developing** I can use a simple comparison to produce True or False.	**Secure** I can choose accurate comparisons for equality, limits and inclusive boundaries.	**Mastery** I can apply the required comparisons accurately across the assessment and explain their effect at exact boundaries.

Logical operators and compound conditions		Secure
Developing I can use a logical operator to connect two conditions.	✓ **Secure** I can use and when all requirements must hold and or when either is sufficient.	**Mastery** I can construct every required compound condition accurately, including overlapping greenhouse conditions and age with consent.

Selection and tracing		Secure
Developing I can follow or create a simple if/else decision.	✓ **Secure** I can use and trace ordered if/elif/else branches, choosing one appropriate result.	**Mastery** I can build and trace all required selection routes accurately, including overlapping thresholds and exact boundary values.

Nested selection		Secure
Developing I can recognise or create a decision inside another branch.	✓ **Secure** I can place a dependent decision inside the appropriate outer branch.	**Mastery** I can organise the required dependent decisions correctly so inner decisions run only in the appropriate outer branch and each else belongs to its intended if.

Validation and invalid routes — Developing

✓ Developing	Secure	Mastery
I can reject a simple invalid value.	I can check the required ranges and allowed options before processing.	I can handle every specified invalid case and prevent invalid data from reaching routing, calculations or receipts.

Debugging and modification — Developing

✓ Developing	Secure	Mastery
I can correct or add a simple condition in supplied code.	I can repair logic or add a route while preserving the main existing behaviour.	I can complete the required repairs and modifications while preserving all specified routes and boundaries.

Inputs, calculations and output — Secure

Developing	✓ Secure	Mastery
I can connect some inputs, decisions and outputs.	I can combine the required inputs and main decisions with calculations and clearly labelled results.	I can meet all assessed input, calculation and output requirements, including accurate membership discounts and complete receipts with money formatted correctly.

Annotation and readability — Developing

✓ Developing	Secure	Mastery
I can add a useful comment to identify part of my program.	I can make the assessed code easy to follow with useful section comments and explanations of important decisions.	I can meet all assessed annotation requirements, explaining threshold priority and the purpose of validation and selection as well as program sections.

Understanding boundary tests — Developing

✓ Developing	Secure	Mastery
I can identify a relevant boundary test.	I can identify the values immediately below and exactly on both thresholds.	I can explain how 59/60 and 79/80 check changes between bands and reveal incorrect comparison operators.

Individual mark checklist

Each criterion shows the teacher decision and its maximum mark.

A1 · Question 01		1 / 1
A1.1	Select 1 <= value <= 12.	1 / 1

A2 · Question 02		1 / 1
A2.1	Select and.	1 / 1

A3 · Question 03	1 / 1

A3.1	Give the output Silver.	1 / 1

A4 · Question 04 — 1 / 1

A4.1	Give the output Consent required.	1 / 1

A5 · Question 05 — 1 / 2

A5.1	Identify that the values are immediately below and exactly on the two thresholds.	1 / 1
A5.2	Explain that the tests check changes between bands and can reveal incorrect comparison operators.	0 / 1

A6 · Question 06 — 1 / 2

A6.1	Write a valid Python comment linked to the Gold threshold.	1 / 1
A6.2	Explain why the check includes 80 and higher scores for Gold eligibility.	0 / 1

B1 · Debug 01 · Age and consent — 0 / 3

B1.1	Prevent approval for ages below 13.	0 / 1
B1.2	Prevent approval for ages above 17.	0 / 1
B1.3	Require consent == "yes" alongside a valid age.	0 / 1

B2 · Debug 02 · Delivery charge — 0 / 3

B2.1	Use delivery £0.00 for an order of £50 or more.	0 / 1
B2.2	Use delivery £3.99 from £20 up to, but not including, £50.	0 / 1
B2.3	Use delivery £6.99 below £20.	0 / 1

Teacher note: Take care with you comparison symbols. > is more than, < is less than.
The syntax is correct here but not the logic.

C1 · Practical 01 · Modify a gaming rank reward — 5 / 6

C1.1	Add Rising for XP from 1000 to 2499 inclusive.	1 / 1
C1.2	Assign 250 reward coins to the Rising route.	1 / 1
C1.3	Retain the correct Starter, Pro and Elite ranks and rewards.	1 / 1
C1.4	Use one if/elif/else chain so exactly one rank is selected.	1 / 1
C1.5	Retain player and integer XP inputs and the labelled player, rank and reward outputs.	1 / 1
C1.6	Explain in a purposeful comment why the highest XP threshold is checked first.	0 / 1

C2 · Practical 02 · Greenhouse ventilation logic		6 / 7
C2.1	Set HIGH when temperature >=30 and humidity >=80, using and.	1 / 1
C2.2	Set LOW when temperature >=30 but humidity is below 80.	1 / 1
C2.3	Set LOW when humidity >=80 but temperature is below 30.	1 / 1
C2.4	Set OFF when neither threshold is met.	1 / 1
C2.5	Use an ordered if/elif/else with HIGH first and or in the LOW condition.	1 / 1
C2.6	Retain the integer inputs and display exactly one final mode labelled Fan mode:.	1 / 1
C2.7	Explain in a purposeful comment why HIGH is checked before LOW.	0 / 1

C3 · Practical 03 · Parcel sorter upgrade		7 / 9
C3.1	Reject weight of 0 kg or less as REJECTED WEIGHT.	1 / 1
C3.2	Reject weight above 20 kg as REJECTED WEIGHT.	0 / 1
C3.3	Prevent rejected weights from entering destination routing.	0 / 1
C3.4	Route national parcels up to and including 2 kg as NATIONAL LIGHT.	1 / 1
C3.5	Route valid national parcels above 2 kg as NATIONAL STANDARD.	1 / 1
C3.6	Preserve LOCAL LIGHT up to 2 kg and LOCAL STANDARD above 2 kg for valid local parcels.	1 / 1
C3.7	Use UNKNOWN DESTINATION for valid-weight parcels with an unrecognised destination.	1 / 1
C3.8	Use nested selection for routing within valid-weight processing.	1 / 1
C3.9	Retain destination and decimal-capable weight inputs and display exactly one labelled Route: result.	1 / 1

Teacher note: The sequencing of you if/elif statements means that your route variable is changed unintentionally.

C4 · Practical 04 · Digital marketplace order		0 / 18
C4.1	Collect customer name, product type and membership with clear prompts.	0 / 1
C4.2	Collect quantity and convert it to an integer.	0 / 1
C4.3	Reject quantities below 1.	0 / 1
C4.4	Reject quantities above 10.	0 / 1
C4.5	Select a unit price of £3.00 for skin.	0 / 1
C4.6	Select a unit price of £8.00 for bundle.	0 / 1
C4.7	Select a unit price of £12.00 for pass.	0 / 1

C4.8	Identify and reject an invalid product.	0 / 1
C4.9	Prevent invalid quantities and products reaching order calculations or receipt output.	0 / 1
C4.10	Calculate subtotal as quantity multiplied by the selected unit price.	0 / 1
C4.11	Calculate a 15% discount when membership is yes.	0 / 1
C4.12	Use £0.00 discount when membership is no.	0 / 1
C4.13	Calculate final total as subtotal minus discount.	0 / 1
C4.14	Identify customer, product and quantity on the receipt.	0 / 1
C4.15	Display labelled subtotal, member discount and final total.	0 / 1
C4.16	Format every displayed money value to two decimal places.	0 / 1
C4.17	Add useful comments identifying input, calculation and output sections.	0 / 1
C4.18	Add purposeful comments explaining validation and product selection.	0 / 1

Control Checkpoint levels are classroom assessment milestones, not GCSE grades. DIRT does not change the original assessment mark.